

Data Structures and Algorithms

Exercise 7: Financial Forecasting

What is Recursion?

Recursion is a programming technique where a method calls itself to solve a problem by breaking it into smaller subproblems. Every recursive method must have:

- **Base Case** – Stops the recursion.
- **Recursive Case** – Calls the same method with a smaller problem.

Example:

- Factorial
- Fibonacci Series
- Tree Traversals
- Financial Forecasting

For financial forecasting, recursion can calculate the future value by repeatedly applying the annual growth rate.

2. Setup

Create a method that calculates the future value using recursion.



Where:

- **FV** = Future Value
- **PV** = Present Value
- **r** = Growth Rate

- **n** = Number of Years

3. Implementation in Java

FinancialForecast.java

```
public class FinancialForecast
{
    static double futureValue(double presentValue, double rate, int years)
    {
        // Base Case
        if(years == 0)
        {
            return presentValue;
        }

        // Recursive Case
        return futureValue(presentValue, rate, years - 1) * (1 + rate);
    }

    public static void main(String args[])
    {
        double presentValue = 10000;
        double growthRate = 0.10; //10%
        int years = 5;

        double result = futureValue(presentValue, growthRate, years);

        System.out.println("Present Value : " + presentValue);
        System.out.println("Growth Rate : " + (growthRate * 100) + "%");
        System.out.println("Years : " + years);
        System.out.println("Future Value : " + result);
    }
}
```

}
}

Output

Present Value : 10000.0

Growth Rate : 10.0%

Years : 5

Future Value : 16105.1

Dry Run

Suppose:

- Present Value = **10000**
- Growth Rate = **10%**
- Years = **3**

The recursive calls are:

futureValue(10000, 0.10, 3)

= futureValue(10000, 0.10, 2) × 1.10

= futureValue(10000, 0.10, 1) × 1.10 × 1.10

= futureValue(10000, 0.10, 0) × 1.10 × 1.10 × 1.10

= 10000 × 1.10 × 1.10 × 1.10

= 13310

4. Analysis

Time Complexity

Each recursive call reduces the number of years by **1**.

- **Time Complexity: $O(n)$**
- **Space Complexity: $O(n)$** (due to the recursion call stack)

where **n** is the number of years.

How to Optimize the Recursive Solution?

The recursive solution creates one function call for each year, which increases memory usage for large values of **n**.

Possible optimizations:

1. Iteration (Loop):

- Replace recursion with a for loop.
- Time Complexity: **$O(n)$**
- Space Complexity: **$O(1)$**

2. Direct Formula:

- Use the compound growth formula:

$$FV = PV \times (1 + r)^n$$

- Using `Math.pow(1 + rate, years)` computes the result efficiently without recursive calls.

3. Memoization (for overlapping recursive problems):

- Store previously computed results to avoid repeated calculations.
- This technique is useful for recursive problems like Fibonacci, though it is generally unnecessary for this simple forecasting calculation.

Conclusion

- Recursion provides a simple and readable way to calculate future values.
- The recursive algorithm has **$O(n)$** time complexity and **$O(n)$** space complexity.
- For practical financial forecasting with many years, using the **compound interest formula (`Math.pow`)** or an **iterative approach** is more efficient because it avoids the overhead of recursive calls.